CheckIt.AI — Rapport d’exploration des sources

Étape 01 — Explorer et qualifier les sources de données
État du projet vérifié le 7 septembre 2026

1. Objectif et périmètre

Ce rapport compare des sources capables de fournir une publication avec du **texte et une image associés**. Le projet reste centré sur un workflow **ETL** : il collecte, contrôle et normalise les données, mais n’entraîne aucun modèle et ne transforme jamais automatiquement une opinion en « fake news ».

Une source est retenue si elle permet de conserver au minimum :

- un identifiant stable ;
- un titre ou un texte ;
- l’URL de la publication ;
- une image accessible et liée à cette publication ;
- la date, la langue et la source ;
- le label original et sa provenance lorsqu’ils existent.

2. Sources explorées

Source	Modalités	Format / accès	Langue	Labels et qualité	Extraction envisagée
NewsData.io	Titre, résumé ou contenu, image, métadonnées	API REST, JSON	Multilingue, dont français	Aucun label vrai/faux	`Requests`, clé API, pagination
PolitiFact	Claim, explication, verdict, vignette, métadonnées	RSS/XML	Anglais	Verdict éditorial documenté : qualité élevée	`Requests` + `Feedparser`
Fakeddit	Titre Reddit, image et métadonnées	TSV + images ou URLs	Anglais	Labels à 2, 3 et 6 classes par supervision distante : qualité moyenne	`csv.DictReader` + `Requests`
The Conversation France	Article complet, auteur, date et image principale	Atom + pages HTML	Français	Aucun label vrai/faux	`Feedparser`, `Requests`, `Beautiful Soup`
FakeNewsNet	Articles, images et données sociales selon disponibilité	CSV, JSON, scripts officiels	Anglais	Labels PolitiFact/GossipCop : qualité variable	Scripts officiels, lecture CSV/JSON
NewsCLIPpings	Image et légende correcte ou falsifiée	JSON + images	Anglais	Bon label de cohérence texte-image, pas un verdict complet	Téléchargement officiel
COSMOS	Image et légendes provenant de contextes différents	Annotations + images	Anglais	Bon label « hors contexte », spécialisé	Téléchargement officiel
Google Fact Check Tools API	Claim, verdict et provenance ; image non garantie	API REST, JSON	Multilingue	Verdict traçable, mais échelles hétérogènes	`Requests`, clé API
Reddit Data API	Posts, images, commentaires et métadonnées	API REST JSON, OAuth	Multilingue	Aucun label fiable natif	API officielle ou PRAW

Les quatre premières sources ont été retenues et réellement intégrées. Les autres restent des pistes d’enrichissement.

3. Sources réellement intégrées

3.1 NewsData.io — API REST

- **Rôle :** actualités récentes, notamment francophones.

- **Extraction :** `scripts/extract_newsdata.py` appelle l'API avec `Requests`, une clé `NEWSDATA_API_KEY`, des filtres et une pagination configurable.
- **Couple multimodal :** un enregistrement n'est conservé que si le texte utile et `image_url` sont présents, puis l'image est téléchargée.
- **Label :** absent ; Transform laisse les trois champs de label à `null`.
- **Limites :** quota du compte, contenu parfois tronqué et image parfois absente.

3.2 PolitiFact — flux RSS

- **Rôle :** publications de fact-checking avec verdict éditorial.
- **Extraction :** `scripts/extract_politifact.py` télécharge le RSS avec `Requests`, puis `Feedparser` lit les entrées, le HTML inclus dans le flux et la vignette.
- **Couple multimodal :** le contenu RSS et `media_thumbnail` appartiennent à la même entrée.
- **Label :** le verdict du Truth-O-Meter est extrait du fragment HTML pendant Transform et conservé avec son système et sa provenance.
- **Limites :** corpus principalement anglophone et flux limité aux publications récentes.

3.3 Fakeddit — dataset TSV

- **Rôle :** lot volumineux de posts multimodaux déjà annotés.
- **Extraction :** `scripts/extract_fakeddit.py` parcourt `multimodal_train.tsv` ligne par ligne avec `csv.DictReader`, filtre `hasImage=True`, puis récupère les images avec `Requests`. Un curseur persistant reprend à la ligne suivante lors du run suivant.
- **Couple multimodal :** l'ID de la ligne sert à nommer l'image locale et à vérifier l'association.
- **Label :** les labels 2, 3 et 6 classes sont conservés sans interprétation ; leur supervision distante implique un bruit possible.
- **Limites :** anglais, URLs parfois anciennes et téléchargement complet des images très volumineux.

3.4 The Conversation France — Atom puis HTML

- **Rôle :** collecter directement des articles français depuis leurs pages HTML.
- **Extraction :** `scripts/extract_theconversation.py` découvre les URLs dans le flux Atom, télécharge chaque page avec `Requests`, puis `Beautiful Soup` extrait le titre, le corps, l'auteur, la date et `og:image`.
- **Couple multimodal :** texte et image principale proviennent de la même page.
- **Label :** absent ; la source n'est pas automatiquement assimilée à la classe « vraie ».
- **Limites :** sélecteurs HTML susceptibles d'évoluer ; nombre de pages volontairement limité avec une pause configurable.

4. Outils utilisés et choix techniques

Outil	Utilisation réelle dans le projet
`Requests`	Appels API, téléchargement des flux, pages HTML et images
`Feedparser`	Lecture du RSS PolitiFact et du flux Atom The Conversation
`Beautiful Soup`	Extraction HTML de The Conversation et nettoyage du fragment HTML PolitiFact pendant Transform
`csv.DictReader`	Lecture progressive du TSV Fakeddit sans charger 149 Mo en mémoire
`pandas` / `pyarrow`	Construction et export du dataset transformé en Parquet

`Scrapy` et `Selenium` n'ont pas été ajoutés : les quatre sources sont accessibles par API, flux, fichier ou HTML statique. Scrapy serait pertinent pour parcourir un grand site ; Selenium seulement si le contenu dépendait réellement de JavaScript ou d'interactions navigateur.

5. Formats et organisation réellement produits

```
data/
├── raw/<source>/extraction_<horodatage UTC>.json
├── images/<source>/<identifiant>.<extension>
├── images/<source>/<identifiant>.metadata.json
├── state/fakeddit_cursor.json
├── processed/publications.parquet
├── processed/publications.jsonl
└── rejected/invalid_records.jsonl

logs/
├── <source>.log
└── transform.log
```

Chaque lot brut est une enveloppe JSON :

```
{
  "source": "newsdata",
  "collected_at": "2026-09-02T08:00:00Z",
  "count": 10,
  "records": []
}
```

Les fichiers `.metadata.json` relient l'URL distante au fichier image local par son identifiant, sa taille et son SHA-256. La phase Transform produit ensuite un contrat commun de **18 colonnes** en Parquet et JSONL, un manifeste reproductible et un journal des rejets.

Le JSON brut horodaté est écrit pendant **Extract** ; les fichiers Parquet, JSONL traité, manifeste et rejets sont écrits pendant **Transform**. Airflow charge ensuite le Parquet dans PostgreSQL et contrôle le résultat.

6. Contrôles qualité et traçabilité

Une publication transformée est acceptée si :

1. son identifiant est présent ;
2. son titre ou son texte est non vide ;
3. les URLs de publication et d'image sont valides ;
4. les dates sont convertibles en UTC et la langue en code ISO à deux lettres ;
5. l'image existe, n'est pas vide et possède une signature JPEG, PNG, GIF ou WebP ;
6. son nom, sa taille, son URL d'origine et son SHA-256 confirment l'association texte-image ;
7. ses trois champs de label sont tous renseignés ou tous absents ;
8. son `publication_id`, ou le couple `source_url + image_sha256`, n'a pas déjà été retenu dans le run.

Les extracteurs utilisent des timeouts, trois nouvelles tentatives sur les erreurs temporaires, des paramètres en ligne de commande et des logs. Chaque tâche transmet par XCom le chemin exact du lot qu'elle vient de créer. Transform ne relit que les lots du run ; PostgreSQL gère l'historique par upsert sur `publication_id`. Pour Fakeddit, le curseur n'est avancé qu'après la sauvegarde réussie du lot JSON ; il empêche de recommencer au début du TSV.

7. Volumétrie et cycle d'acquisition

Chaque run produit exactement un lot par source avec une volumétrie comparable :

Source	Publications demandées par run
NewsData.io	10
PolitiFact	20
Fakeddit	20
The Conversation France	10
Total nominal	**60**

Transform exige les quatre chemins du run courant et refuse un lot manquant, dupliqué ou issu d’une source inconnue. Pour Fakeddit, le curseur mémorise la prochaine ligne du TSV et n’avance qu’après la sauvegarde du lot. PostgreSQL cumule ensuite les publications par upsert, sans dupliquer la clé primaire. Aucun enregistrement n’est accepté sans preuve complète reliant son URL à son image locale.

8. Conclusion

La sélection finale couvre quatre modes d’acquisition complémentaires : API JSON, RSS, fichier TSV et HTML. Elle conserve la relation texte-image, la provenance et les labels disponibles. Le contrat transformé est chargé dans PostgreSQL par le DAG Airflow.

Références principales

- Multimodal Fake News Detection: A Survey
- NewsData.io — documentation
- PolitiFact — flux RSS
- Fakeddit — dépôt officiel
- The Conversation France